可重构脉动阵列矩阵加速器

OpenNPU-Lite

团队 Hextech 编号 41971

摘要

边缘智能计算既需要低精度算力，也需要把有限的片上存储、数据搬运和主机软件组织成可用的计算通路。针对小型 FPGA 上矩阵尺寸多变、分块供数成本高和应用接线复杂的问题，本项目基于 PYNQ-Z2 构建 OpenNPU-Lite 可编程矩阵加速平台。系统采用 8×8 物理脉动阵列，以 64×64 逻辑分块完成 K 维归约，通过 K 优先（K-major）布局、输入预取和双 C-bank 调度组织计算与结果输出，并以链式任务接口支持运行时矩阵几何配置。ARM 负责分块构帧、资源管理及应用量化，PL 承担规则矩阵乘加，硬件计数器与独立整数参考共同验证执行行为。

完整实现工作于 100 MHz，使用 26,762 LUT、27,575 FF、11 BRAM Tile 和 72 DSP，setup 与 hold 时序均收敛。512^3 任务的纯 PL 有效吞吐为 4.467 GMAC/s；保持硬件不变的两轮主机优化分别将各自同场 512^3 完整请求缩短 10.76%和 9.54%。应用侧完成固定 12 窗 FPGA Pointwise—ARM Depthwise—FPGA Pointwise 真实共享会话，768,000 个标量检查点全部一致；完整 KWS 混合图则在 PC 离线环境中从公共 WAV 动态执行至分类结果。团队将接口设计、代码级修订、异机构建与现场验证贯通，并把失败案例提炼为验证 Skill，形成包含源码、构建报告、实测原件和分层复现入口的工程交付。

关键词：FPGA；INT8；脉动阵列；软硬件协同；关键词识别

第一部分 作品概述

1.1 功能与特性

OpenNPU-Lite 在同一比特流上提供通用 INT8 矩阵乘加与关键词识别 Pointwise 计算接入。主机在规定尺寸内配置 M、N、K，将任务分成逻辑块并通过 AXI DMA 连续提交；PL 返回 INT32 累加结果，ARM 完成应用所需的零点校正、偏置、逐通道重定量化及激活。系统支持尾块填充、跨 K 面板累加和链式任务执行，配备工作量及周期计数器。驱动将资源

初始化、数据传输、返回校验与释放分开管理，固定 12 窗应用链可在一次资源会话中复用 Overlay 和两块根缓冲区。独立整数参考、传输帧逆解码和返回原件回读支持逐层追溯。

1.2 应用领域

平台面向资源受限的边缘矩阵计算与嵌入式智能原型验证，适合将规则的低精度点积交给 FPGA、将控制与复杂逐通道算术保留在处理器的任务。项目以通用 GEMM 和 DS-CNN 关键词识别建立两类应用入口，考察矩阵几何变化、主机组织与跨层量化对计算通路的影响。与模型专用的固定电路相比，运行时任务描述便于在既定阵列和比特流上接入多种形状；与单独展示计算核相比，完整请求测量进一步回答主机和搬运成本是否抵消阵列收益。

1.3 主要技术特点

物理阵列包含 64 个 PE，同一阵列分时处理 64×64 逻辑输出块，并在 C-bank 中归约 K 面板。K-major 布局按阵列供数顺序组织字节，同一子任务内预取下一面板；首 K 面板覆盖初值，双 C-bank 让计算与已完成结果输出交叠。主机采用请求内面板复用、平坦构帧（flat）及连续面板物化，保持线格式不变地减少事务组织成本。应用接口以算子、张量和窗口身份约束动态数据流，原始点积 INT32 先于量化核验；一次性子任务准入、父子进程监督和同一锁文件索引回读连接正常完成与资源释放。

1.4 主要性能指标

表 1　主要性能指标

指标	结果
PL 时钟	100 MHz
LUT/FF/BRAM/DSP	26,762/27,575/11/72
setup/hold 余量	+0.118/+0.066 ns
512^3 纯 PL 有效吞吐	4.467 GMAC/s
10/22 请求改善	512^3 9.54%；256^3 10.32%
真实共享 KWS 中段	12 窗，768,000 检查点一致

1.5 主要创新点

（1）以 K-major 供数、面板预取与双 C-bank 组织小型阵列连续执行。（2）以完整请求分解定位主机瓶颈，在固定硬件与线格式上优化软件；两轮独立同场比较的 512^3 请求分别缩短 10.76%和 9.54%。（3）以动态张量包络和单共享会话接通 PL—PS—PL 链，固定 12 窗 768,000 个检查点零差异，并联合核验原始点积、传输帧与生命周期。

1.6 设计流程

团队确定算术、接口与验收条件，指导模块编码和脚本实现；先以仿真及整数参考核算术，再完成异机完整构建、受控板测和原件复算。性能分解决定优化对象，失败反例推动函数级修订。应用从单层映射推进至动态共享中段和完整离线图，最后按固定版本整理来源、Skill 及复现入口。

第二部分　系统组成及功能说明

2.1 整体介绍

项目围绕“有限阵列怎样承担可变化的矩阵任务”展开设计。脉动阵列通过相邻 PE 传递操作数并复用部分和，适合规则点积；Gemmini 的全栈接口、Eyeriss 的数据复用和 NVDLA 的分层计算组织为本项目提供了机制参照[1–3]。本项目的设计重点是把这些思想落实到 XC7Z020 的资源规模与 Python/ARM 主机环境，连接逻辑分块、线格式、驱动和实际请求。

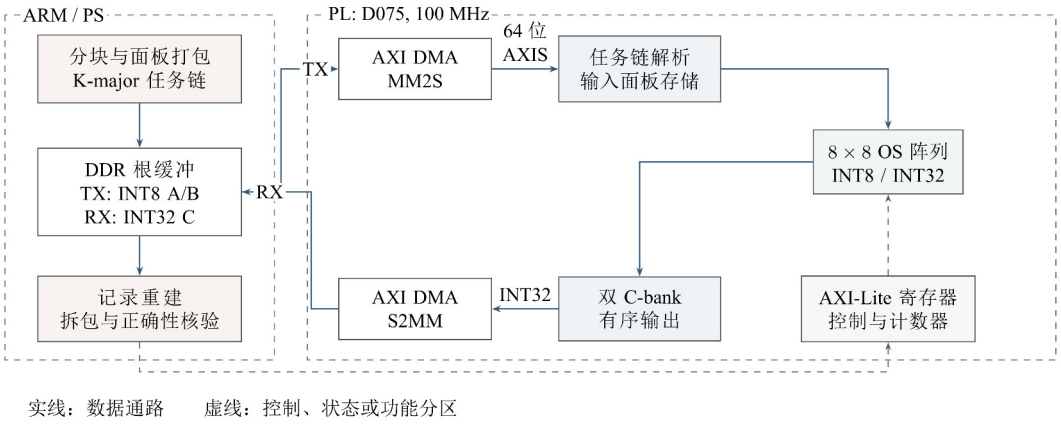


图 1　ARM、DDR 缓冲、AXI DMA 与 PL 计算核组成请求通路；实线为数据流，虚线为控制或状态。

图 1 给出系统通路。ARM 将任务映射到 DDR 中的连续物理缓冲，AXI DMA 把请求送入 PL；控制寄存器管理启动、模式与状态，结果沿接收通道回到 DDR。PL 内部完成流解析、操作数供给、阵列计算及 C-bank 输出。应用层读取返回值后执行校正和量化，或重建通用 GEMM 结果。项目采用本地 PC 工具与板端 ARM 协作，不依赖云端推理服务。

软硬件划分依据是计算规则性和接口成本。密集 INT8 点积交给 PL，窗口选择、张量几何、逐通道尺度及生命周期管理交给 ARM。矩阵接口返回 INT32，既能直接核对原始点积，也便于不同模型在同一硬件上接入各自的量化参数。功能划分见表 2。

表 2　软硬件划分与关键接口

部分	主要职责	关键接口
PC 工具	版本核验、构建、原件复算、报告分析	清单 SHA、构建回执、归档
ARM 主机	分块构帧、共享资源、校正与应用调度	矩阵/张量包络、DMA 缓冲
AXI DMA	DDR 与 PL 之间的流式搬运	64-bit AXI-Stream、TLAST
PL 计算核	解帧、供数、点积、累加与输出	AXI-Lite 状态、INT32 流
独立核验	原始点积、传输帧、计数与释放核对	INT64/标量参考、原件

2.2 硬件系统介绍

2.2.1 硬件整体介绍

平台采用商用 PYNQ-Z2，器件为 Zynq-7000 XC7Z020。处理系统与可编程逻辑通过 AXI 互连，PL 工作时钟 100 MHz。自研部分包括流式矩阵核、输入组织、C-bank 控制、链式任务解析及性能计数；板载 DDR、以太网及电源电路沿用开发板。完整实现资源与自然 setup/hold 结果在第三部分给出。

设 A 为 M×K、B 为 K×N，矩阵结果为 C。系统执行 C[i,j]=Σ A[i,k]B[k,j]。INT8 操作数扩展到累加通路，输出保存为 INT32；软件参考采用 INT64 或 Python 无界整数逐值复算。在最大 64×64 的逻辑输出块内，8×8 物理阵列遍历 64 个微块，每个 PE 计算当前 K 面板的局部部分和，C-bank 保存跨 K 面板的归约结果。

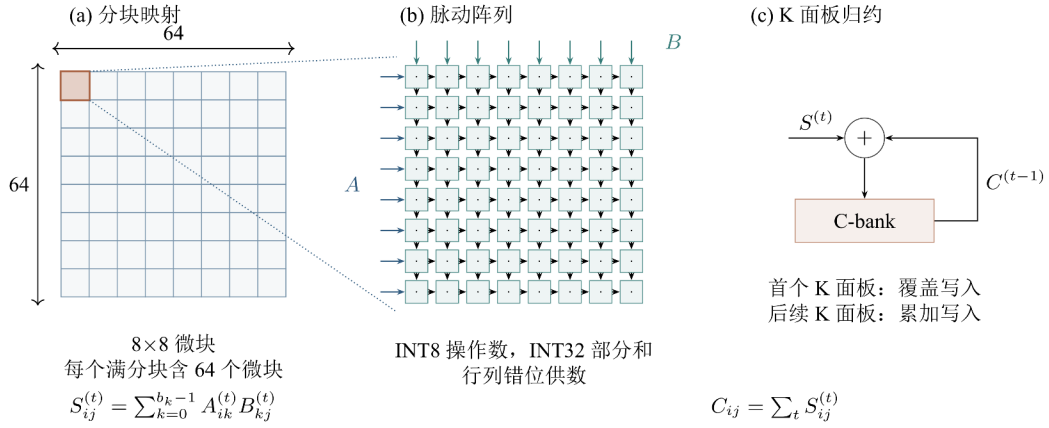


图 2　64×64 逻辑块由同一 8×8 物理阵列分时计算；PE 产生局部部分和，C-bank 跨 K 面板归约。

物理 PE 数量由硬件参数确定；主机运行时配置矩阵几何和任务序列。通用分块路径覆盖合同内 1..512 的维度，应用入口进一步限定合法形状。非满块按线格式填充并只取有效结果，例如 KWS 的 125×64 输出分为 64 行与 61 行两块，尾部 3 行补零。

2.2.2 机械设计介绍

作品采用开发板与外接测量仪器集成，无自研机械传动、机壳或结构件。现场按既定单路 USB 供电及 J9 选择连接，网线用于主机与板端通信；能量实验通过 UT70 记录供电口读数。实物图 1–2 展示供电链及板卡端口。

2.2.3 电路各模块及关键输入输出

K-major 将同一 K 位置的通道（lane）连续排列，使 64-bit 流字能够对应阵列供数。满面板按分组、K 索引和通道排序，接收侧把字节送入对应输入存储。第一 K 面板覆盖 C 的初值，后续面板累加，减少额外清零步骤。同一子任务执行期间预取下一 K 面板，面板边界再切换输入存储。

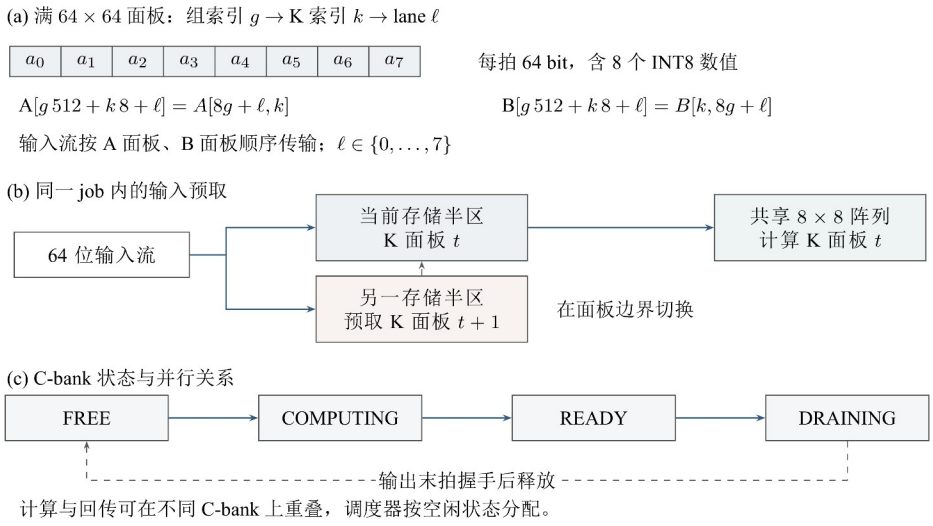


图 3　K-major 供数、子任务内预取与双 C-bank 状态关系。时序图说明逻辑依赖，不按实测周期比例绘制。

双结果存储体（C-bank）将结果存储分为两组。一组计算时，另一组可输出已完成块；存储体依次经历空闲（FREE）、计算（COMPUTING）、就绪（READY）和排空（DRAINING），最后一拍握手完成后才释放。图 3 表达逻辑依赖，具体交叠由运行计数核对。输出背压会影响存储体回收，因此控制器必须同时处理计算完成与 AXI-Stream 握手。

AXI DMA 使用简单 DMA（Simple DMA）通路[4]。主机先安排接收，再提交发送，分别核长度、容量和完成状态。输入链中的帧头描述子任务编号与几何；输出为按合同排列的 INT32 矩阵。针对 125×64×64 的单层调用，两个子任务（job）共 16,512 B 输入、32,000 B 输出及 512,000 MAC。控制与状态回读采用 MMIO，运行期 MODE11 核对 d007_global_work_v2、版本 2、error_code=0、status=5 以及 failed_tile_index=65535。清理软复位后处于尚未启动的版本 1 身份，核验器按阶段分别检查。

图 4 给出本工程的原生连接关系。GP0 与 ctrl_smc 承载 PS 对 DMA 及加速器的控制；DMA 的流接口连接加速器，存储器主接口经 mem_smc 接入 HP0。时钟、复位和中断连接同时保留，便于从驱动调用追查到实际接口。[E01、E41]

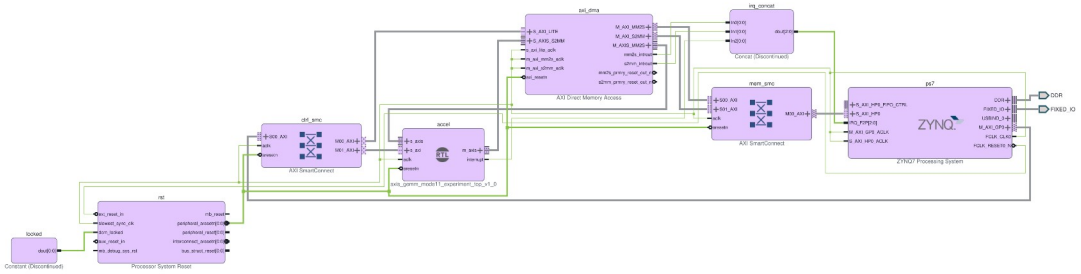


图 4　Vivado 原生 Block Design。PS 通过 GP0 和 ctrl_smc 访问 DMA 及加速器控制寄存器；DMA 的 MM2S/S2MM 流接口连接加速器，存储器主接口经 mem_smc 访问 PS 的 HP0。FCLK 及 Processor System Reset 提供时钟/复位，中断经 irq_concat 汇入 PS。原生 SVG/PDF 随本版保留。[E01、E41]

2.2.4 RTL 流水与接口验证

图 5 将 8×8 阵列的输入、乘积和累加放在同一时间轴上。固定用例采用 M=N=8、K=1，两块激活分别为 1 和 2、权重行为 1 至 8，输出行分别为 1 至 8 和 2 至 16。PE(0,0)在 1005 ns 接收输入，随后两拍产生乘积和累加；PE(7,7)更晚响应，直观呈现阵列传播与流水关系。[E41]



图 5　8×8 阵列的 Vivado 原生仿真波形。固定测试台以 M=N=8、K=1 驱动真实计算核心；图示首块中，激活沿阵列传播，乘积与累加随后更新。PE(0,0)的输入、乘积和累加分别在 1005、1015 和 1025 ns 出现，PE(7,7)对应时刻为 1145、1155 和 1165 ns。时钟周期 10 ns。[E41]

图 6 显示 bank1 计算写入与 bank0 结果读回同时进行，重叠计数从 0 增长到 16；图 7 放大输出握手，显示背压期间的数据保持和有效握手后的计数推进。用例共核对 2 块任务、64 个输出拍及局部复位行为，fail=0。[E41]

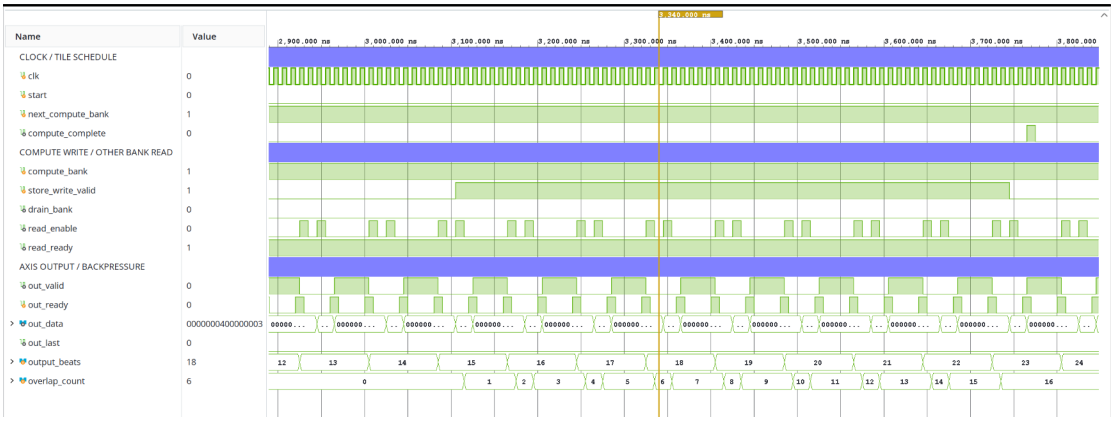


图 6　双 C-bank 计算写入与结果读回的原生波形。约 3105–3735 ns 期间，计算结果写入 bank1，bank0 的读回仍在进行；计数器记录 16 次不同银行的 PE 写入与有效读回重叠事件。两块用例共输出 64 个 64-bit 拍，2 次计算完成，fail=0。[E41]

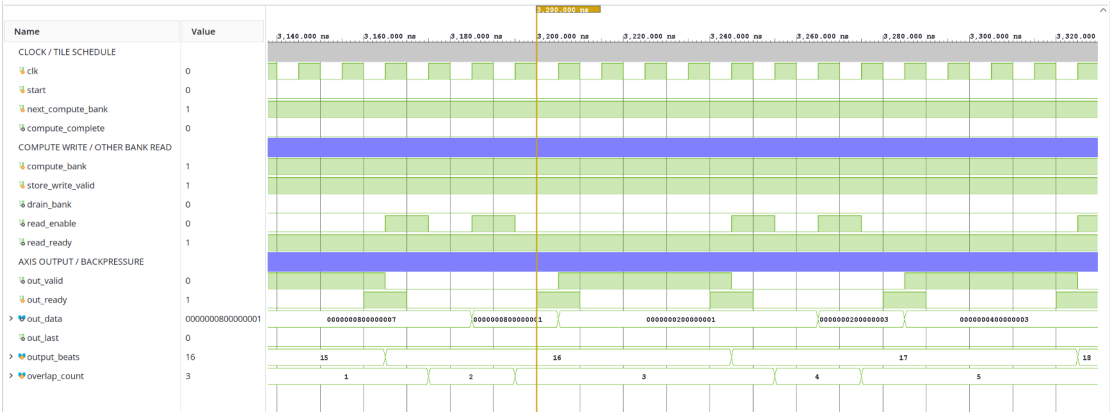


图 7　AXI-Stream 输出背压局部。约 3210–3240 ns 内 out_valid=1、out_ready=0，out_data 保持 0000000200000001；有效时钟沿握手后 output_beats 才推进。图中 read_enable/read_ready 属于 C-bank 读接口，out_valid/out_ready 属于输出接口，两组握手分别解释。[E41]

本轮固定测试台重放中，PE 单元 30 项、AXI 包保护 21 项检查通过。历史 MODE11 随机回归另保留 31 组情形与 31,002 次元素配对，MMIO 回归读到 D075 构建身份，独立重定量化流水保留 1,016 项检查。各用例的原始日志和本轮波形来源分别记录。[E41]

2.3 软件系统介绍

2.3.1 软件整体介绍

软件分为通用矩阵、KWS 应用和复现工具三层。通用矩阵路径生成分块与任务链，共享资源管理对象持有 Overlay 及根缓冲，调用对象通过驱动提交。KWS 把 Pointwise 转成 GEMM，ARM 执行逐通道校正与 Depthwise；完整离线图继续连接 WAV 特征、CPU 前后

缀和分类。工具层对源码、BIT/HWH、输入名单及原件建立尺寸和 SHA 索引，以固定版本复跑。

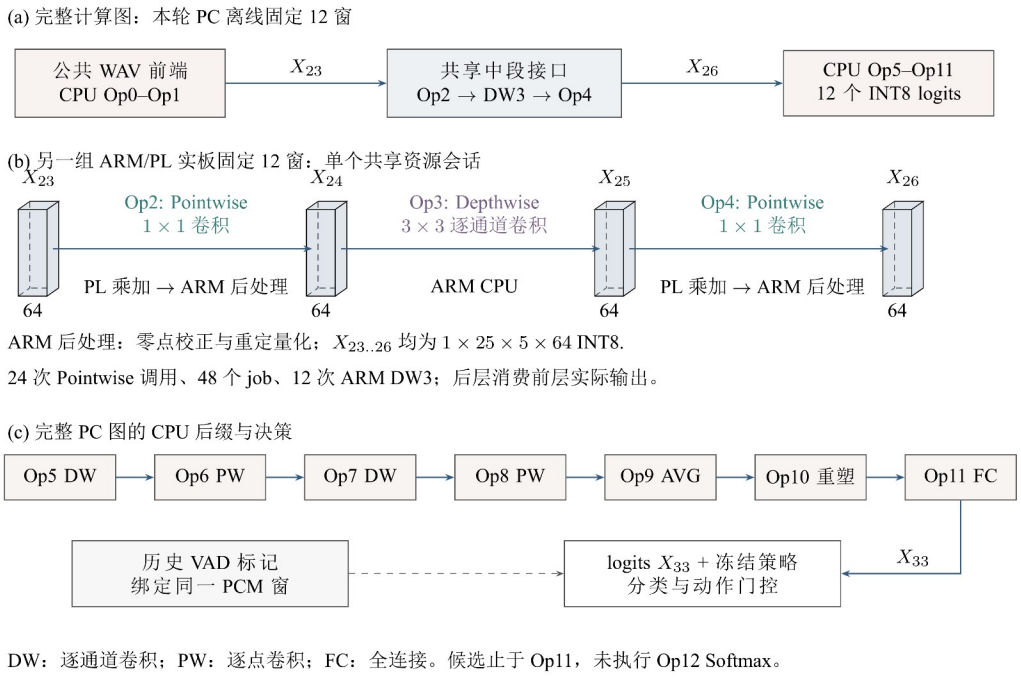


图 8　完整 PC 离线图与真实共享 12 窗中段的对应关系。真实中段输入来自冻结 CPU 前缀；PC 图从 WAV 动态计算，PL 采用软件夹具。

DS-CNN 采用 Depthwise 与 Pointwise 交替结构[8]。Op2 与 Op4 同为 125×64×64，但权重、张量和前驱不同。真实共享中段按 tensor23→Op2→tensor24→DW3→tensor25→Op4→tensor26 执行。DW3 把动态 Op2 输出变形为 1×25×5×64 的 NHWC 数组，以输入零点−128 完成 3×3 SAME 填充，逐通道点积并重定量化，再生成独立拥有内存且 C 连续的新矩阵供 Op4 使用。

完整 PC 图从公共 WAV 生成 49×10 个 INT8 特征，CPU 执行 Op0、Op1 形成 tensor23，共享接口执行中段后，由 CPU 运行 Op5..11 得到 12 个 Softmax 前分类值（pre-softmax logits）。该图的 PL 资源由 WireEnvironment 夹具模拟：夹具解码实际发送字节后计算返回，黄金张量只用于事后比较。真实共享中段与完整 PC 图的运行环境分别记录，图 8 对应两者的数据依赖。

2.3.2 软件各模块介绍

矩阵主机先检查数据类型、形状和配置，再按 M/N/K 生成块及首子任务编号。请求内重复使用的面板先构造一次，flat 路径直接写入平坦帧；连续面板路径使用 ascontiguousarray(view).tobytes()物化 C 序视图[6]，发送字节与原路径保持一致。满面板至

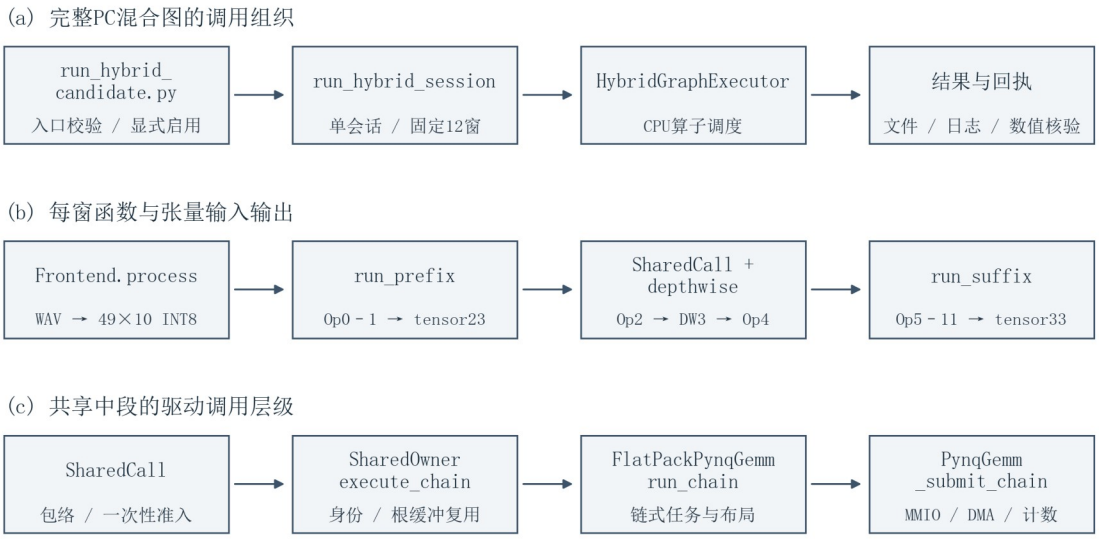
多新增 4,096 B 连续临时数组，这一数值描述单面板临时分配，不代表进程峰值驻留内存（RSS）。

驱动调用链从 SharedCall 进入 SharedOwner.execute_chain，经 FlatPackPynqGemm.run_chain 调用 PynqGemm._submit_chain。一个 SharedDeviceSession 只初始化一次 Overlay、分配两块根缓冲并在正常结束后关闭一次；24 次交替请求使用一次性 JobPermit，对应连续子任务编号 1..48。PYNQ 的物理连续内存和缓存同步按其分配接口管理[5]。资源管理对象遇到返回前异常时保留未决资源，父进程通过实际等待退出和进程组查询记录退出，避免在硬件状态未明时释放缓冲。

InputEnvelope 绑定算子、张量、窗口、来源及完整值，在构造和提交前拒绝错层、跨窗或不独立拥有内存的输入。返回后先核 8,000 个原始点积 INT32，再执行输入零点校正、偏置及逐通道定点重定量化，最后核 INT8。量化使用冻结模型的尺度与零点，逐通道权重规则参照 LiteRT 规格[7]。此顺序可以识别被 ReLU 截断为−128 的单比特错误；传输帧逆解码则确认下游实际收到当前 DW3 输出。

函数流程按调用层级和实际数据依赖展开。完整 PC 候选由 run_hybrid_candidate.py 核对输入后调用 run_hybrid_session；HybridGraphExecutor.run_prefix 接收 Frontend.process 现算的 49×10 INT8 特征，执行 Op0、Op1，输出独立拥有内存的 125×64 tensor23。共享中段返回 tensor26 后，run_suffix 继续执行 Op5 至 Op11，生成 12 个 INT8 分类值 tensor33。

SharedCall 接收绑定窗口和张量身份的 InputEnvelope，经 SharedOwner.execute_chain、FlatPackPynqGemm.run_chain 进入 PynqGemm._submit_chain。每个 Pointwise 请求的有效输入帧为 16,512 B，返回 32,000 B 原始 INT32；返回后核对 8,000 个值，再完成偏置、零点校正和 INT8 重定量化。DW3 接收 Op2 的动态输出 tensor24，生成 tensor25 供 Op4 使用。上述函数流中，完整 PC 图使用 WireEnvironment；已完成的真实 ARM/PL 共享中段采用同一驱动调用层级，现场输入及执行范围见 KWS 验证章节。[E21、E30–E34]



补图 1　上两行描述完整 PC 混合图，第三行展开共享中段调用。图中输入输出由实际候选源码核对；完整 PC 图与真实共享中段的运行环境分别记述。

报告修订使用 scientific-writing 组织来源与章节核对，配合 anti-defensive-writing 和 lieflat-less-ai-tone 检查叙述；技术交互按来源核对，新增说明另行审校。相关程序化写作工作流参考 Scientific Agent Skills[9]，这些工具用于文稿与来源整理。

2.3.3 团队主导的实现与验证 Skill

团队将设计目标落实到函数调用、数组布局与测试出口，通过实现指导、代码审查和调试参与驱动及脚本迭代，编程模型辅助完成候选代码和反例。参考绑定任务明确覆盖 derive、调用对象构造、make_jobs 及执行入口四处 sources()调用，并将冷绑定参考与受限错误阶段纳入修订。跨层接线由团队确定 Op2→DW3→Op4 路线，把前层实际返回值、NHWC 变形和实际传输帧核验写入实现要求。[E27、E40]

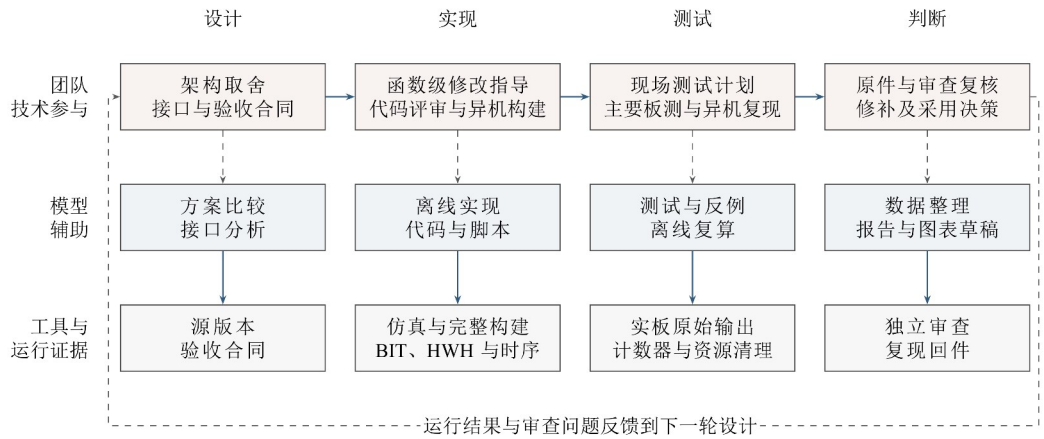


图 9　团队组织设计、函数级实现修订、测试与取舍；编程模型辅助编码与分析，异机构建和现场实验形成反馈。

协作案例以技术任务和实现修订呈现。参考绑定任务由团队给出函数范围和执行顺序，模型辅助完成绑定模块与顺序回归；DMA 身份检查的初步方案依赖实例名，团队结合伪 DMA 反例复核实现，修订为精确 MODTYPE/VLNV 声明检查，并补充 HWH-only 与 CLI 回归。阶段计时分析则经团队复核起止点，移除 transfer/wait 的重复归因。任务要求、候选方案、审查修订及验证结果构成连续的工程协作记录。[E27、E40]

pynq-dma-validation 先核 BIT/HWH 和精确 IP 声明，再检查 DMA 几何；pynq-gemm-validation 先核布局和原始点积，再核量化结果；engineering-evidence-review 从计时起止与样本定位结论。检查器在 Linux 通过 90 个方法且 0 跳过，复现成员在另一电脑以 AI 辅助使用 Skill 处理十个既有合成案例，按事前 40 项标准评阅为 40/40。前者验证程序行为，后者检验既定案例中的使用过程。图 9 呈现团队设计、实现评审、异机构建与现场验证之间的反馈。

团队把审查问题写成具体实现任务。附录中的完整输出接线和四处参考调用清单，展示了要求如何进入代码；模型首轮引用错误及 ticket 采集次序的修正，则保留了实现自检与补测过程。原文和修订结果见文末技术交互附录及[E43]。

2.3.4 分层复现入口

工程以 opennpu_lite_41971 组织 src、sim、build、board、data、skill、report 及 provenance。顶层 manifest 索引文件数量以本交付 manifest 为准，外部 SHA256 固定为下值。取得完整目录后先核身份，结果写到包外新目录；文档随附《复现操作指南》，列出环境安装、完整命令、预期输出及错误处置。

<外部 checksums.txt 中的 manifest_sha256>

PC 已测环境为 Windows x86-64、Python3.12.14、NumPy2.3.5。先用可信 Python 创建独立 venv 并安装 numpy==2.3.5；在 PowerShell 填写实际路径后执行以下入口。每一行命令退出非零即停止，保留标准输出、标准错误及运行回执；输出目录本身必须不存在。

```
$basePy = 'C:\Python312\python.exe' # 实际可信 Python3.12 路径
$root = 'D:\OpenNPULite\opennpu_lite_41971' # 完整工程已解压
$envDir = 'D:\OpenNPULite\python_env_01'
$runs = 'D:\OpenNPULite\runs'
function Invoke-Checked([string]$Exe,[string[]]$ArgList) {
  & $Exe @ArgList
  if ($LASTEXITCODE -ne 0) { throw '命令失败，停止并保留输出。' }
}
if (Test-Path -LiteralPath $envDir) { throw '环境已存在，另选新目录。' }
Invoke-Checked $basePy @('-m','venv',$envDir)
$py = Join-Path $envDir 'Scripts\python.exe'
Invoke-Checked $py @('-m','pip','install','numpy==2.3.5')
if (-not (Test-Path -LiteralPath $runs)) {
  New-Item -ItemType Directory -Path $runs | Out-Null
}
$pin = '<外部 checksums.txt 中的 manifest_sha256>'
Invoke-Checked $py @('-I','-B',"$root\reproduce.py",'--manifest-sha',$pin)
Invoke-Checked $py @('-I','-B',"$root\reproduce.py",'--manifest-sha',$pin,
  '--profile','pc','--enable-offline','--output',"$runs\pc_01")
Invoke-Checked $py @('-I','-B',"$root\reproduce.py",'--manifest-sha',$pin,
  '--profile','hybrid-smoke','--enable-offline','--output',"$runs\smoke_01")
Invoke-Checked $py @('-I','-B',"$root\reproduce.py",'--manifest-sha',$pin,
  '--profile','hybrid12','--enable-offline','--output',"$runs\hybrid12_01")
```

默认文件核验（check）应返回 files 与交付清单一致且不执行推理。PC 回执核实际子进程退出 0、运行后文件不变标记 immutable_export_verified_after=true；Windows 旧入口 5 项平台跳过单列。混合图核窗口名单、全值差异及历史 VAD 标记。原候选 12 窗独审与导出一窗验证分别留存，运行者用自己的新输出确认结果。

仿真将 sim/d075 复制到独立可写目录，按 sim/run_regression.ps1 指定 VivadoRoot 执行 xvlog、xelab 和 xsim。完整构建采用 Windows Vivado2026.1、Zynq-7000 器件/IP 支持与 PYNQ-Z2 板定义，依次运行 src/hardware/verify_kit.py、rebuild.py check、prepare、build；板定义 SHA、工具参数及命令见随附指南。成功需取得资源、自然 setup/hold 报告和 BIT/HWH/XSA，不能只检查进程退出。成员已完成同配置 CMD 路径的异机完整构建；新 v3 包装本身的真实冷构建尚待接收者验证。

板端复做从 board/quickstart.md 取得当前运行包和完整步骤，先核原件、时钟、供电及真实环境，再生成新的预检记录和运行计划；操作者确认后单次运行，依次只读回读、归档及两端 SHA 核对。历史启动标识、锁索引和运行计划用于回读，不复制成新现场值。公开仓库地址和第三方资产再分发许可在发布前另行核定；本次文档制作不启动仿真、构建或板测。

2.4 设计取舍与验证依据

设计取舍同时考察正确性、映射资源与完整请求。早期 C-store 流水写回方案通过 15 个测试台的 4,196 项检查，OOC 时序也满足 100 MHz，但 Vivado 把 4096×32 存储映射为 1,408 个 RAM64M，而非预期的 4 个 RAMB36E1。团队因此撤回该方案，恢复明确的 BRAM 基线，并将端口行为和银行所有权纳入后续设计。这一过程说明，仿真、综合和集成实测分别揭示不同工程约束。[E26]

硬件稳定后，团队用请求分解选择下一优化对象；接口修订先形成可执行反例，再由完整调用与返回原件核对。共享 KWS 将已验证矩阵接口连接到动态张量，Skill 将身份、算术和测量规则提炼为可复用步骤。复现按 2.3.4 的环境、输入身份、软件、仿真、构建和板端层次推进，每层检查实际输出并保留失败。

第三部分　完成情况及性能参数

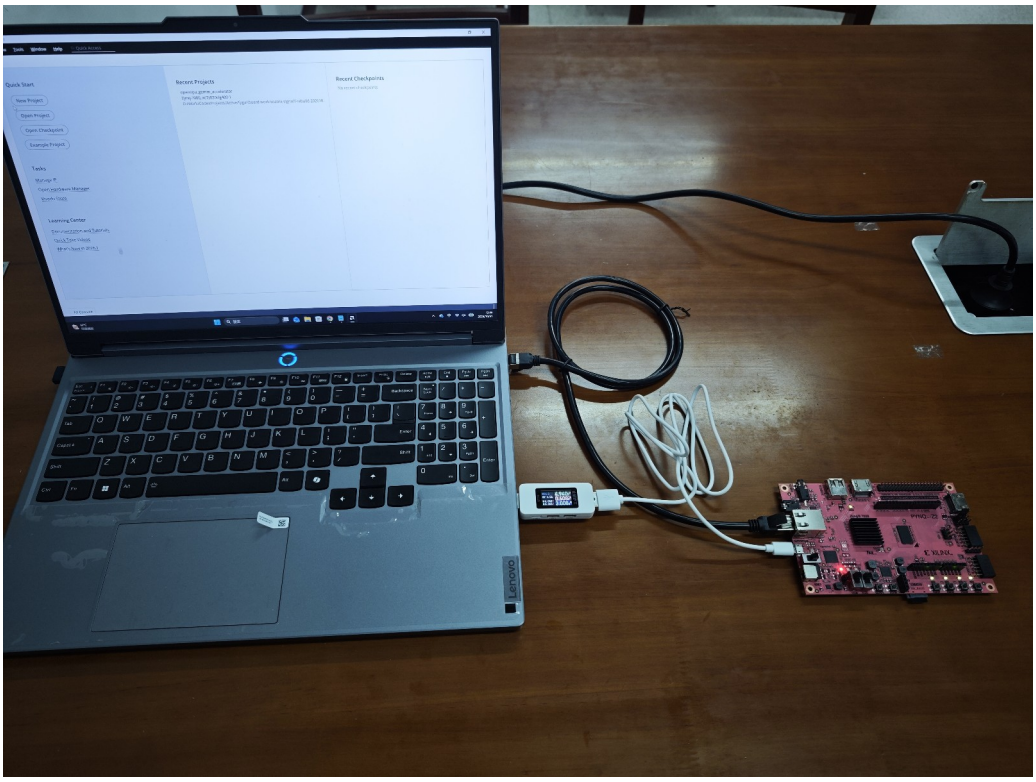
3.1 整体介绍

系统已形成 PYNQ-Z2 硬件、ARM 运行软件、PC 复现工具与 UT70 采集分析的完整实验平台。矩阵任务验证计算通路，KWS 验证模型张量跨 PS/PL 传递，原始回读、清单与独立参考连接软件输出和真实设备行为。

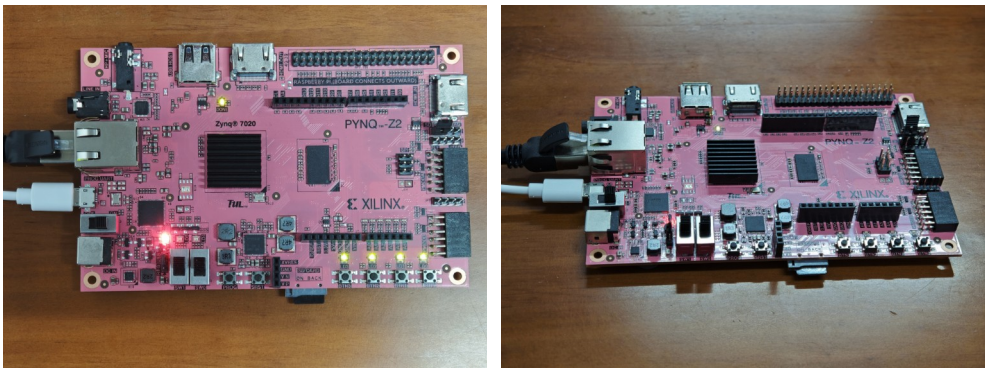
实物图 1–2 展示系统连接、开发板俯视与斜视，供电与通信路径可与 2.2 节的接口说明对照。

实物平台与连接

实物平台由主机、PYNQ-Z2 开发板与 USB 测试仪组成。以太网承担板端通信，单路 USB 路径提供供电；俯视和斜视照片展示主要接口、散热器及指示灯。



实物图 1　系统全貌。主机经以太网连接 PYNQ-Z2，USB 供电路径串接 UT70 测试仪。



实物图 2　开发板实物。（a）俯视；（b）斜视。网口与 Micro-USB 接口位于板卡左侧，独立 DC 输入未连接。

3.2 工程成果

3.2.1 机械成果

采用商用开发板，无独立机械结构件。全貌照片承担实物组合展示，避免以结构示意代替实际装置。

3.2.2 电路成果

D075 完成综合、布局布线、BIT/HWH 生成与实板运行，主实验 BIT 和 HWH 绑定同一工程。交付保留 RTL、约束、完整实现报告及原始运行归档。实物图 2 的开发板近照对应 2.2 节的实物接口；自研逻辑关系由图 1–3 及源码说明。

最终硬件以源码与生成物身份固定。D075 在十月继续用于矩阵和共享 KWS 中段；10 月 13 日干净重建核对 47 份输入，资源与自然时序报告正文一致，4,045,564 B 配置载荷逐字节相同。构建成员在另一电脑完成同配置 CMD/BAT 完整构建，并补齐来源端 ZIP 哈希。[E01、E18、E19、E21]

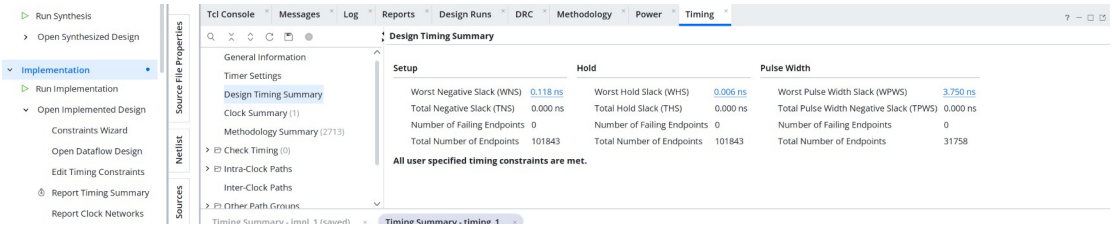


图 10　同一 D075 路由检查点的原生时序窗口。此视图保留实现阶段额外 0.060 ns 最小延迟要求，WNS=+0.118 ns、WHS=+0.006 ns，TNS/THS 均为 0；恢复原 0 ns 要求后的自然约束签核报告为 WHS=+0.066 ns。两份原报告分别保留。[E01、E41]

图 10 为额外 0.060 ns 最小延迟要求下的实现视图，WHS 为+0.006 ns；表中自然约束签核值为+0.066 ns，两者按约束分别解释。厂商 DDR preset 相关告警、DMA RAM Advisory 及 152 条 TIMING-46 逐路径核查随工程保留。[E01、E41]

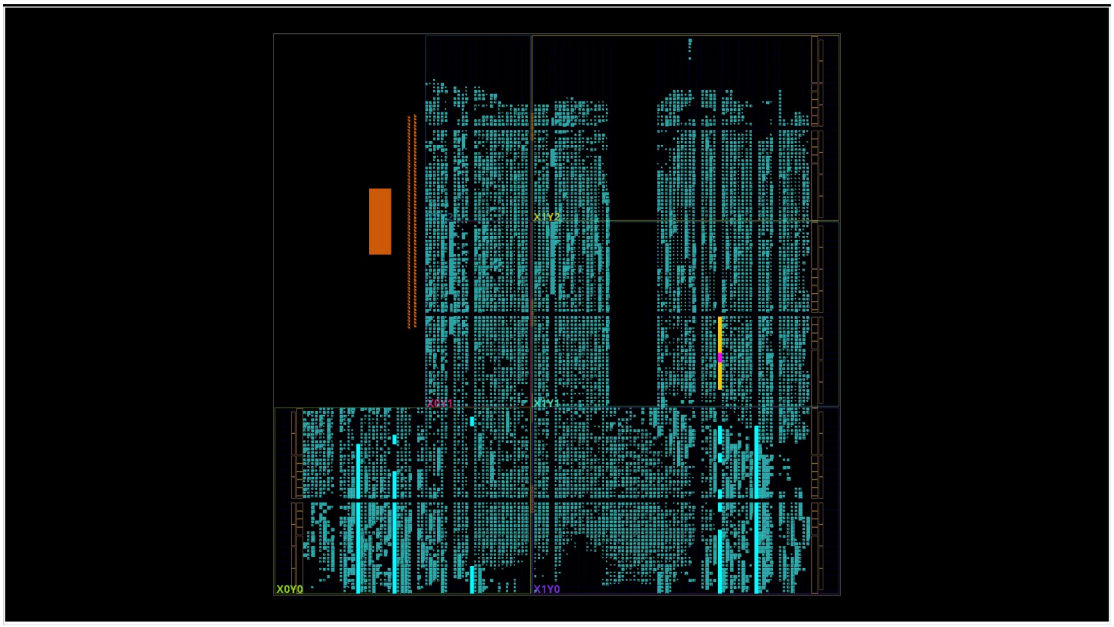


图 11　D075 已实现设计的 Device 资源分布。按已放置网表逐项核对，64 个 PE 乘法 DSP 以青色高亮，7 个量化 DSP 为橙色，1 个工作量计数 DSP 为紫红色，72 个物理 DSP 站点互不重复。背景中的青色已放置逻辑同时包含 LUT/FF 等资源，DSP 用途以对象与站点清单判定。[E01、E41]

图 11 结合物理站点清单说明 72 个 DSP 的用途分配。随附 PE 原理图则从实际数据连接选取单 PE 乘法器的实现局部，保留 SVG/PDF 供放大查阅。[E41]

3.2.3 软件成果

软件交付包括矩阵主机、共享 KWS 候选、整数参考、清单核验、只读回读与复现入口。运行图 1–3 展示矩阵与共享中段的实际完成状态和结果汇总，实物图 3 补充同次演示的系统斜视画面。完整工程及报告的来源表记录各资产的版本、大小和 SHA。[E42]

现场运行与完成状态

2026 年 10 月 31 日，平台在同一次开机中顺序完成矩阵对照与共享 KWS 中段演示。矩阵部分执行 72 次调用，其中 24 次预热、48 次正式；共享中段执行 12 窗、24 次 Pointwise 请求和 48 个 job。两段启动器均以 0 退出，并保存正常清理记录。运行图 1–3 取自此次原始录屏。[E42]


```
{"schema": "threeway-outer-exit-candidate-v1", "actual_exit_code": 0, "actual_wait_observed": true, "proc
ess_group_absent": true, "operation_success": true, "automatic_kill": false, "automatic_retry": false, "p
romotion": false, "run_ready": false, "group_query_injected": false, "stop_seen": false, "wait_intervals"
: 202, "source": "Popen.wait then read-only process-group query", "child_receipt": {"board_access": true,
 "diagnostic_complete": true, "energy_evaluated": false, "error_type": null, "fixture": false, "operation
_success": true, "os_evidence_observed": true, "os_facts": {"actual_successful_calls": 72, "actual_waited
_children": 12, "all_cleanup_verified": true, "all_groups_absent": true, "canonical_lock": {"device": 458
26, "inode": 214963, "path": "/tmp/opennpu-kws-chain.lock"}, "formal_calls": 48, "warmup_calls": 24}, "ou
ter_process_exit_observed_by_this_receipt": false, "performance_accepted": false, "physical_identity_auth
enticated": false, "promotion": false, "request_sha256": "30f4e4950b5cdc22868f2ded4523ca8fd3b46db2896a59d
5ac23acbd6e8c4f79", "run_ready": false, "schema": "threeway-round-candidate-receipt-v1", "secondary_error
_types": [], "signals": {"handler_restore_errors": [], "stop_signals": []}}, "artifact_identity": {"files
": 78, "manifest_sha256": "c60a3da42f0f681d36c7b4d3f777248a8128864a3de27af76c65bc72d737ae64"}, "release_r
eadback": {"device": 45826, "inode": 214963, "path": "/tmp/opennpu-kws-chain.lock", "lock_released_same_i
node": true, "source": "lstat same inode + no matching FLOCK in proc/locks; no acquisition probe"}, "sign
als": {"stop_signals": [], "handler_errors": [], "handler_restore_errors": [], "original_supervisor_uncha
nged": true}}
ACTUAL LAUNCHER EXIT=0
MATRIX_RECORDED_EVIDENCE_CHECK_PASS
EXISTING RECEIPT DISPLAY - no inference, no live lock access
```


运行图 1　矩阵测试完成画面（原始录屏 04:15，局部）。回执记录 72 次成功调用、48 次正式调用及资源清理通过，末尾显示启动器退出码 0 和记录核验通过。


```
ion_quality_evaluated": false, "cleanup_verified": true, "energy_evaluated": false, "error_type": null, "
evidence_injected": false, "finite_supervisor_fixture": false, "fixture": false, "group_query_injected":
false, "last_stage": "validate_terminal", "live_ticket_observed": true, "lock_released_same_inode": true,
 "operation_success": true, "original_infinite_hold_executed": false, "parent_go_sent": true, "parent_gua
rdian": {"held": false, "lease_retained": false, "pid": 9710, "root_buffers_retained": 0, "runner_retaine
d": false, "safety_unresolved": false}, "performance_evaluated": false, "process_group_absent": true, "pr
ocess_group_proof_real": true, "promotion": false, "real_DMA_executed": true, "real_cross_layer_PL_execut
ion": true, "real_lock_access": true, "release_readback": {"device": 45826, "inode": 214963, "lock_releas
ed_same_inode": true, "path": "/tmp/opennpu-kws-chain.lock", "source": "lstat same inode + no matching FL
OCK in proc/locks; no acquisition probe"}, "request_sha256": "a767128a5a07ab1a85e862bb82f8f8384c363bc13f6
5262b8cda6c4c18cf819f", "run_id": "84d68a88-4642-4781-af43-c0875c815f95", "run_ready": false, "signals":
{"handler_errors": [], "handler_restore_errors": [], "original_supervisor_unchanged": true, "stop_signals
": []}, "source": "Popen.wait then read-only process-group query", "sources_unchanged": true, "stop_seen"
: false, "wait_intervals": 222}
ACTUAL_SHARED_LAUNCHER_EXIT=0
```


运行图 2　共享中段完成画面（原始录屏 09:39，局部）。终态记录真实 DMA、真实跨层 PL 执行、同 inode 锁释放及清理通过，启动器退出码为 0。

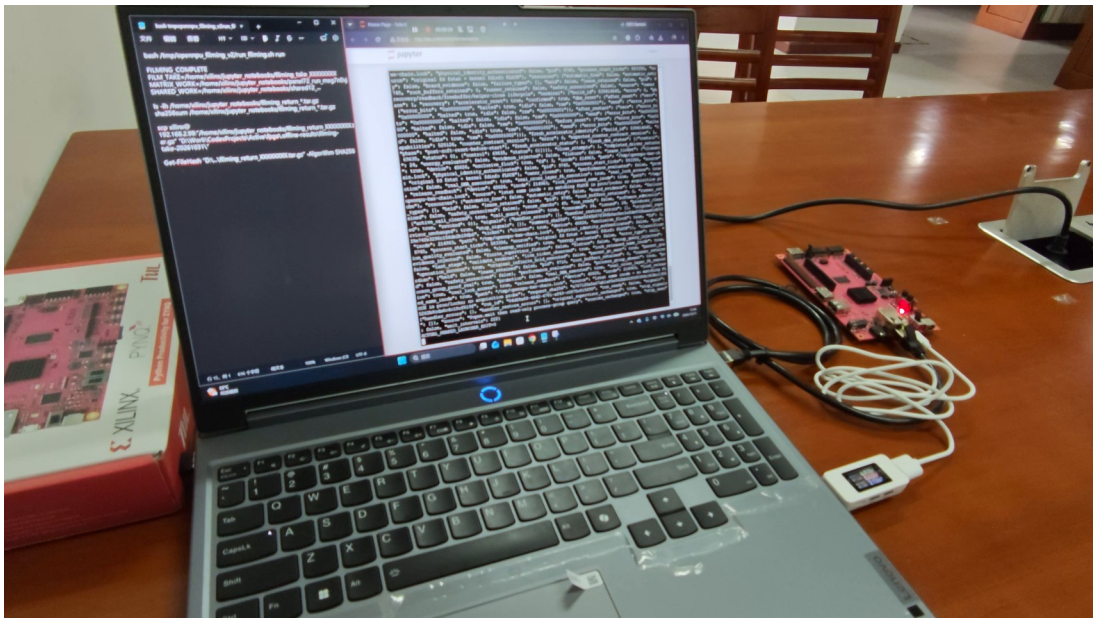
两类任务的返回原件、进程退出观察和清理记录共同对应此次演示。图中回执字段保持原样，关键结果可与附录 E42 中的原始 JSON 逐项核对。

复现结果与现场装置

本次矩阵完整请求均值在 256^3 下由 57.601 ms 降至 51.742 ms，在 512^3 下由 261.115 ms 降至 238.188 ms，分别缩短 10.17%和 8.78%；同场双线程 OpenBLAS 均值为 25.073 ms 和 166.295 ms。该组独立复现与 10 月 22 日结果方向一致，按当日场次单列。[E42]

```
OPENNPU FILMING RESULT | existing evidence readback
MATRIX: 256^3 / 512^3; 72 calls = 24 warmup + 48 formal
Fixed representative outputs and 72 recorded checks verified by frozen reader.
256^3  fpga_flat_baseline     mean_ms=57.601  n=8
256^3  fpga_flat_contiguous   mean_ms=51.742  n=8
256^3  openblas_2t            mean_ms=25.073  n=8
512^3  fpga_flat_baseline     mean_ms=261.115  n=8
512^3  fpga_flat_contiguous   mean_ms=238.188  n=8
512^3  openblas_2t            mean_ms=166.295  n=8
Timing scope: full FPGA request / CPU wrapper; not pure-PL throughput.
SHARED: Op2 -> ARM DW3 -> Op4; 12 windows / 24 requests / 48 jobs
Recorded golden bytes verified; normal exits and cleanup verified.
READBACK ONLY: no inference rerun, no classification/energy/performance claim.
FILM_TAKE=/home/xilinx/jupyter_notebooks/filming_take_YI529Wv3
```

运行图 3　完成后的只读结果汇总（原始录屏 10:10，局部）。六组计时各含 8 次正式调用，末段汇总 12 窗 / 24 请求 / 48 个 job；计时范围为完整 FPGA 请求或 CPU 包装入口。



实物图 3　连续演示现场的系统斜视画面。主机、PYNQ-Z2、UT70 及 USB 供电和以太网通信连接同时入镜，取自手机原始录像 09:28。

共享中段逐字节核对 Op2 的 raw / 校正 / INT8、DW3 的累加器 / INT8、Op4 的 raw / 校正 / INT8，共八类 768,000 个检查点。此次记录用于复现数值返回、请求排程与正常退出；分类质量和能量结论沿用对应独立实验。录屏时间点和原件 SHA 见 E42。

3.3 特性成果

3.3.1 资源、时序与硬件执行

表 3 比较 D007-P 和 D075 两套完整集成实现。D075 新增结果存储及相关控制，代价为 2,473 LUT、2,525 FF 及 4 BRAM Tile；DSP 数量保持 72，其中 64 个用于 PE、7 个用于量化 1 个用于计数。该表描述集成版本之差，各机制的独立贡献由对应实验解释。[E01、E35–E37]

表 3　完整集成实现资源比较

资源	D007-P	D075	XC7Z020 容量
LUT	24,289	26,762	53,200
FF	25,050	27,575	106,400
BRAM Tile	7	11	140
DSP	72	72	220

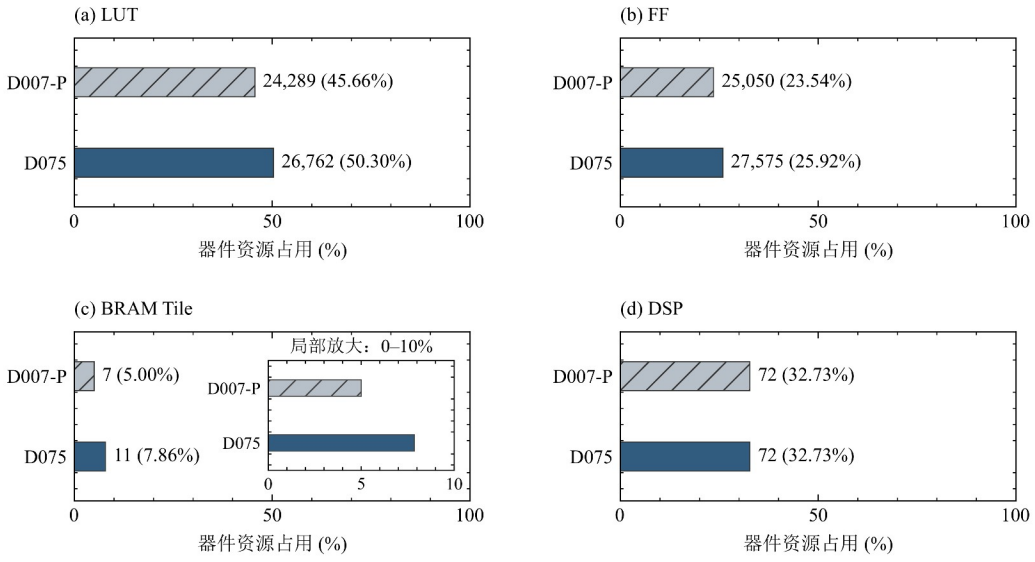


图 12　D007-P 与 D075 完整集成实现的资源比较，按 XC7Z020 容量归一化；BRAM 附局部放大。

100 MHz 完整实现的 setup 余量+0.118 ns、TNS=0，hold 余量+0.066 ns、THS=0。512^3 纯 PL 时间 30.044830 ms，按 134,217,728 MAC 计得 4.467249 GMAC/s；同类 DMA+PL 窗口

为 4.293349 GMAC/s。64 个 PE 在 100 MHz 下理论峰值 6.4 GMAC/s，有效吞吐达到理论峰值的 69.80%，这一比值按整个纯 PL 窗口计算。

同一 BIT 内比较 MODE10 与 MODE11 的 20 对 512^3 任务，DMA+PL 从 36.221124 ms 降至 31.261776 ms，约 1.159 倍。预取和双 C-bank 分别面向输入等待与输出阻塞，计数守恒检查与墙钟结果共同验证调度作用。[E01、E05]

资源利用与数据复用需要联合评价。当前 11 BRAM Tile 占器件 7.86%；512^3 任务按输出子任务重复发送面板，输入载荷为 4,194,304 B，相对原始 A、B 合计 524,288 B 为 8 倍。一次请求中 64 个 A 面板和 64 个 B 面板各自独立打包，传输帧内共出现 1,024 次面板。原始输入、原始输入加 INT32 输出、实际传输帧输入加输出三个边界对应算术强度约 256.0、85.33 及 25.58 MAC/B。跨子任务操作数驻留是后续明确优化对象，当前结果反映既有协议下的调度与主机改进。[E35、E37]

3.3.2 主机优化与完整请求

两轮实验固定 D075 硬件，分别比较旧构帧与 flat、flat 与连续面板。每场共 72 次调用，24 次预热、48 次正式，按尺寸与后端分组后每条件 8 个正式样本，使用两组固定输入和交替执行顺序。CPU 对照均为同场双线程 OpenBLAS。表 4 给出完整请求均值，百分比只在同行基线与候选之间计算。[E07–E11、E38、E39]

表 4　完整请求均值及同场 CPU 对照

场次/尺寸	FPGA 基线 ms	FPGA 候选 ms	同场 CPU ms	请求变化
10/21　256^3	57.296	57.693	25.163	+0.69%
10/21　512^3	272.113	242.822	172.241	−10.76%
10/22　256^3	58.733	52.672	25.342	−10.32%
10/22　512^3	266.977	241.510	171.555	−9.54%

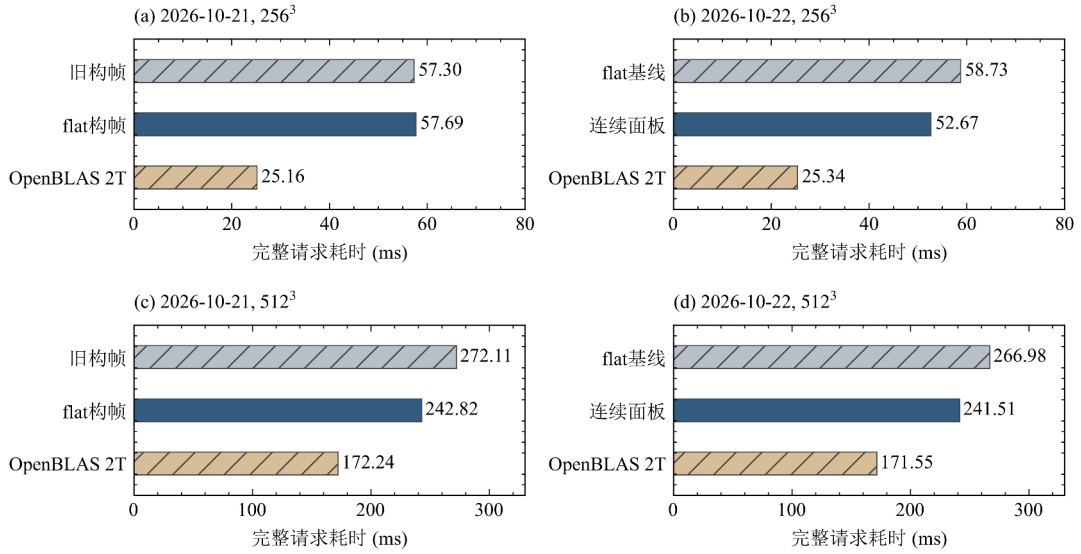
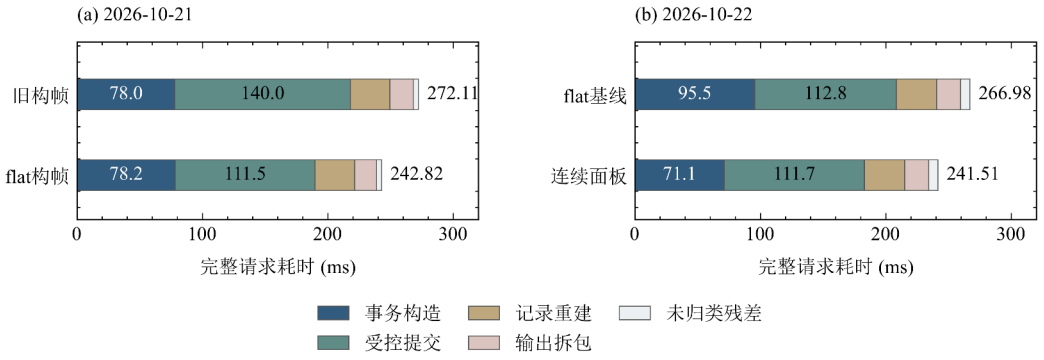


图 13　两场独立实验的完整请求及各自同场双线程 OpenBLAS 对照，每条件 8 个正式样本。

10/22 的 512³ 事务构造从 95.454 ms 减至 71.131 ms，而内部 DMA+PL 为 30.849/30.966 ms，表明本轮收益主要来自主机组织。图 14 将完整请求拆为互斥外层阶段及残差，内部 DMA+PL 作为受控提交的细节，不重复相加。两轮 512³ 分别改善 10.76%和 9.54%，对应不同日期和基线；完整 FPGA 请求仍长于各自同场 CPU。



各阶段均值 (ms)，显示值四舍五入。

条件	事务构造	受控提交	记录重建	输出拆包	未归类残差	合计
10/21 旧构帧	77.95	139.97	31.47	18.62	4.09	272.11
10/21 flat构帧	78.16	111.55	31.69	17.40	4.02	242.82
10/22 flat基线	95.45	112.83	32.18	19.13	7.38	266.98
10/22 连续面板	71.13	111.72	32.09	19.22	7.35	241.51

图 14　512³ 完整请求的互斥外层阶段与残差。DMA+PL 位于受控提交内部，不重复计入总量。

优化采用依据是全请求实测。native_prepared 候选曾在实板退化 47.07%，团队保留失败并停止后续长测，将工作集中到已定位的构造成本。该过程说明，减少高层调用或转向原生代码需要结合实际数据组织验证。[E08]

3.3.3 同工作量能量观察

能量实验采用一组 CPU—FPGA—FPGA—CPU 的 ABBA 次序，每窗 452 次正式调用，预热在保存前完成。UT70 上位机以 10SPS 保存完整包络，按 Wh 端点和同源功率积分分别核算。CPU 两窗能量均值 262.6938 J、FPGA 两窗 258.9786 J，端点差 −3.7152 J（−1.4143%）；功率积分差−1.3989%。两种核算基于同一次采样，图 15 保留四窗原值及差额。[E12、E13]

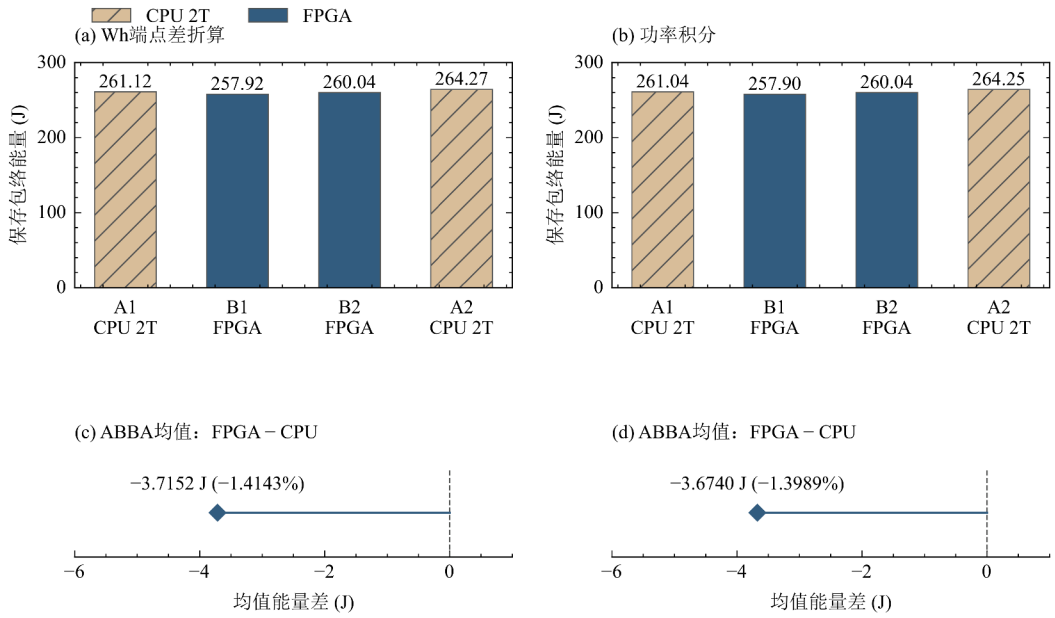


图 15　一组 ABBA 四窗完整保存包络，每窗 452 次正式调用。Wh 端点与同源功率积分分别核算，无单次调用折算或误差条。

完整保存包络包含相应服务过程，未扣空闲，也不折算单次推理能量。同场请求均值 CPU 为 170.5267 ms、FPGA 为 220.4783 ms；进程 CPU 时间从 331.8055 ms 降至 195.3539 ms。此次结果呈现了延迟、处理器占用与供电能量的不同变化方向。稳定节能评价还需重复配对和独立单位/精度核定，尤其需确认 CSV 原始 mV/mA 标头与按 V/A 解释的换算关系。

3.3.4 KWS 应用接入与动态等价

ARM CPU420 从公共 WAV 重新计算前处理和完整 CPU 整数链，205,800 个特征值及 5,040 个分类值与 PC 参考逐字节一致。新 DS-CNN 分类正确 383/420（91.19%），旧模型为 350/420；改善主要来自 unknown，目标口令正确数为 268/300，对照旧模型 270/300。按冻结门控，新模型正确目标放行 214/300，负例误动作 0/120、错误目标口令放行 1 条。固定集分类和动作门控分别评价，当前门限保持原值。[E14、E16]

真实共享中段在一次 SharedOwner 会话中执行 12 次 Op2、12 次 ARM DW3 及 12 次 Op4，共 48 个连续子任务。Pointwise 工作量 12,288,000 MAC，DW3 含空间填充逻辑工作量 864,000 MAC。Op2 与 Op4 各核原始点积、校正、INT8 三类，DW3 核累加器与 INT8 两类，每类 96,000 值，合计 768,000 个检查点零差异；264 份二进制对应冻结参考。24 条运行 MMIO 记录满足 MODE11 版本 2、65535 哨兵与工作守恒，父进程等待退出、进程组消失、一次关闭及同一锁文件索引释放闭合。[E21、E22]

完整 PC 图固定 12 窗进一步完成 5,880 个动态特征和 144 个动态分类值对齐。纯标准库逐层复算 865,680 个算子输出，737 份二进制跨平台逐字节一致；17 项单测 0 跳过、5 项 CLI 负例按预期拒绝。动态结果放行 8 个正确命令，两个已有错分类因低分拒绝，unknown 与 silence 被非命令规则拒绝。评分采用 Op11 真实输出，VAD 使用绑定本窗 PCM 的历史标记。[E30–E34]

表 5　三组 KWS 验证的覆盖关系

验证层次	实际环境与输入	完成内容
CPU420	ARM；公共 WAV	前处理与完整 CPU 整数参考
共享 12 窗中段	真实 ARM/PL；冻结 CPU 前缀 tensor23	动态 Op2→DW3→Op4 及正常清理
完整图 12 窗	PC；公共 WAV 与 PL 软件夹具	前端、前后缀、共享接口及分类

3.3.5 验证范围与后续条件

当前矩阵优化解决了已测任务的主机开销，进一步降低外存流量需调整跨子任务驻留与调度。KWS 端侧参考平均每窗前处理 2,031.107 ms、模型 1,160.126 ms，两段合计 3,191.233 ms/一秒音频，尚不满足实时输入；完整混合模型实板与 420 窗混跑质量是后续独立验收对象。完整 PC 图保留前置清单常量和测试用 VAD 替代参数的两项非阻塞观察，固定运行入口不开放该替代参数。历史 MODE11 长稳记录为 1,800.008108 s、8,302 对运行，

对应原硬件和入口；共享 12 窗验证正常生命周期，性能、恢复与长稳按各版本分别评价。[E05、E29–E34]

第四部分　总结

4.1 可扩展之处

下一步以已定位成本为导向：为跨子任务的 A/B 操作数驻留设计地址与调度接口，降低重复传输；在保持线格式和全值核验的条件下优化主机事务组织；将完整 KWS 图接入真实 ARM/PL 并评估前端、VAD 与目标口令门控。各项改进沿用同场完整请求、固定数据集和独立数值参考，分别评价延迟、质量与能量。现有任务接口、计数器和分层复现工具为这些扩展提供可比较的基线。

4.2 心得体会

研发中最重要的转变，是把优化对象从阵列扩展到完整请求。硬件窗口缩短后，事务构造仍占较大比例。团队据此保持已验证硬件，细化主机数据布局和调用路径，以同场实验决定采用。实测让设计取舍有了共同尺度。

原生主机路径是一次关键取舍。它在 14 次功能冒烟中完成 737,224 个 INT32 位置精确核验，但七种形状的首次完整调用均变慢，512^3 从 283.486 ms 增至 416.921 ms。团队核对返回、计数与源码后，发现新路径仍包含原驱动工作，并增加输入摘要、帧快照和操作数比较。局部构帧收益没有覆盖整条请求的成本，团队因而停止正式长测，保留功能成果与负结果，将后续优化转向面板组织。[E08]

CPU 对照也影响优化判断。团队引入同场双线程 OpenBLAS，以固定输入、预热和交替次序比较完整请求，再用互斥阶段分解定位改进。纯 PL 吞吐、DMA 与主机耗时分别回答不同问题，最后由请求级数据决定方案取舍。这个过程使优化围绕可定位的成本展开。[E07、E38、E39]

团队对实现的参与贯穿接口设计、函数级指导、代码审查和测试修订。负责人将参考绑定细化为四处 sources()调用与错误出口，将跨层组合落实为动态张量、NHWC 变形及实际发送帧检查。构建成员在另一电脑编排 CMD/BAT 构建入口，完成完整构建与来源哈希核对；复现成员使用 Skill 回核案例。负责人组织接线、计划核对与主要板测，10/26 共享链采用授权的浏览器辅助执行并保留人工确认。模型辅助代码、脚本和反例，团队据运行反馈作出修订与采用判断。

几次失败直接改善了方法。伪 DMA 实例迫使检查器从名字判断转向精确声明；量化掩盖的单比特反例使原始点积检查前移；计时重复归因促成互斥外层分解。团队将这些经验写成 Skill，通过程序回归和异机使用确认规则的作用范围。项目由此交付了可编程矩阵通路、真实 PS/PL 应用中段和完整离线计算图，也形成了从问题、编码修订到实测取舍的协同研发过程。

第五部分　参考文献

[1] GENC H, et al. Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration[C]. DAC, 2021. https://arxiv.org/abs/1911.09925v3 . 工程：https://github.com/ucb-bar/gemmini .

[2] CHEN Y H, KRISHNA T, EMER J, SZE V. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks[C]. ISSCC, 2016: 262–263. https://www.mit.edu/~sze/eyeriss.html .

[3] NVIDIA. NVDLA Hardware Architectural Specification, v1[EB/OL]. https://nvdla.org/hw/v1/hwarch.html .

[4] AMD. AXI DMA PG021: Direct Register Mode (Simple DMA)[EB/OL]. https://docs.amd.com/r/en-US/pg021_axi_dma/Direct-Register-Mode-Simple-DMA .

[5] PYNQ. Allocate[EB/OL]. https://pynq.readthedocs.io/en/latest/pynq_libraries/allocate.html .

[6] NumPy. ndarray.tobytes; ascontiguousarray[EB/OL]. https://numpy.org/doc/1.25/reference/generated/numpy.ndarray.tobytes.html ; https://numpy.org/doc/stable/reference/generated/numpy.ascontiguousarray.html .

[7] Google. LiteRT 8-bit quantization specification[EB/OL]. https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization_spec .

[8] ZHANG Y, SUDA N, LAI L, CHANDRA V. Hello Edge: Keyword Spotting on Microcontrollers[EB/OL]. arXiv:1711.07128, 2017, v3 修订于 2018-02-14. https://arxiv.org/abs/1711.07128v3 .

[9] KASSIS T, AGARWAL V, HE Y, et al. Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents[EB/OL]. arXiv:2609.00065v2, 2026. https://doi.org/10.48550/arXiv.2609.00065 .

上述技术文档及相关工作核查日期为 2026-10-27。E 编号对应工程报告来源索引；随附来源对照表记录本系统版各节复用的原件与 SHA，复现命令详见随附《复现操作指南》。

E42 10 月 31 日连续演示原件与原始录屏截图，见随附 evidence/e42.md 及 run_source_index.json。

附录　团队主导的人机协作记录

团队从源码核查和测试结果中选择问题，提出接口级要求，并据此组织实现与验收。以下三例依据实际交互整理，对措辞和重复内容作编辑性归并；后续发现按实际先后记录。原始节选及索引单独保留，便于核对。

C01　先弄清计时边界，再决定改哪里（2026-10-20）

负责人汇总源码核查与审查意见，把优化问题落实到事务构造、构帧和记录重建。模型随后收紧差额的解释，避免把嵌套计时差额全部归到一个函数。

团队提示词

先检查事务构造、pack_chain 和记录重建，找出每次请求里重复创建对象、遍历内容或拷贝数据的位置。对照源码核准各段计时的起止点，再提出保持计算和验证语义不变的小改动。旧驱动保留，新方案和旧方案、当场 OpenBLAS 双线程按同一完整请求口径比较。

模型对归因边界的修正

约 88.7 ms 是两个嵌套计时窗口之间的差额

模型安排的实现步骤

先读相关调用链，再提出最小改动，不直接重写驱动。

团队据此以互斥外层阶段和嵌套窗口组织诊断。10/21 的 flat 构帧与 10/22 的连续面板分别进行同场验证，实测结果见性能章节；不将两个场次连乘。来源：E17、E27、E38、E39。

C02　把首错保留落实到函数参数（2026-10-21）

负责人采纳并提交代码级接线意见，要求把完整输出连同摘要交给协调器。该指令明确了下一次请求能否启动的判断位置，也让反例能够检查失败数据是否真正留存。

团队提示词

输出返回后，先计算 digest = hashlib.sha256(output.astype('<i4', copy=False).tobytes()).hexdigest()，把(spec, result, digest, output)一起交给 RoundCoordinator.accept，让内部 verify_output 核对完整输出。要让 RetainedOutputs 在纯数值不匹配时也能留下首份.bin，核对完成后再允许下一次请求。旧、新后端沿用 runner_type 切换，硬件计数器和安全检查保持原样。

模型的答复与判定修正

我会按前两点推进第四阶段的离线接线，保持旧驱动、安全门禁和冻结包不变。第三点会拆成两层判定：正确性、退出与清理是硬门；性能按同场实测单独报告。“同量级”或优于历史负结果不能预先算性能通过，拷贝量也不直接换算成收益。

模型对全量核对顺序的落实

已确认三后端比较层已有旧 / 新 runner_type 选择逻辑，我会沿用它。接下来补 FPGA 块执行器的逐次握手和父进程退出观察，让每次输出先进入统一全量核对，再允许下一次请求；同时验证纯数值错误能留下首错输出。

接线候选完成 306 项历史离线测试及 12 个真实 PC 子进程演练。首错输出、停止与退出观察分别验收；该阶段的硬件、MMIO/DMA 和规范锁均为软件夹具。来源：E17、E38；matrix-threeway-runtime-20261021_01。

C03　四处调用覆盖与两次实现修正（2026-10-24）

实板失败诊断把问题指向参考模块的冷绑定与运行期导入顺序。负责人采用诊断方案，进一步提交四处调用点及受限错误阶段的清单。下面按原时间顺序呈现这一次要求及模型连续反馈，包含首轮测试发现的问题和封包复查后的补测。

团队提示词

请把四处 sources()调用都查到：worker 派生、调用对象构造、make_jobs 和执行入口。参考模块在导入 pynq 之前先校验、绑定，owner 启动后使用这份绑定，保留对 tensorflow、tflite_runtime 及同名异路径模块的检查。原数学、计数、监督和锁合同不变。补齐冷绑定后 pynq 持续驻留的 12 窗完整顺序测试；报错时写清发生阶段，诊断只留有界 ASCII 摘要，不含环境变量、凭据或未受限栈帧。

11:07 UTC　模型确认实现范围

我会按已批准范围实现新版本，覆盖你列出的四处运行期调用点，并补齐阶段诊断和初始化后的 12 窗顺序测试。原包与失败原件保持不变；完成离线验证后整理交审，不启动板测。

11:11 UTC　模型完成第一版接线

实现已把运行时参考函数放入独立命名空间，原离线函数及其禁止 pynq 的保护保持不变。运行时只有在核对已初始化 owner 与其持有的 PYNQ 模块对象一致后才允许调用参考函数。接下来验证四处调用点，并检查原函数的代码和全局绑定没有被改写。

11:15 UTC　首轮测试发现引用问题并修正

新增的 20 项测试已全部通过，包括持续保留模拟 pynq 模块时的 12 窗完整调用，以及第 4 窗数值故障、参考构造故障和调用构造故障后的清理检查。首轮测试也暴露并修正了一个模块引用可能间接保留 owner 的问题，失败日志已保留。现在继续跑既有回归；这些仍是电脑上的软件夹具，不算实板通过。

11:25 UTC　封包复查发现采集顺序问题

完整离线检查已通过，正在复核实际打包的 ZIP。复查资源路径时还发现一个需要补强的细节：owner 初始化成功后，应先保存原锁 ticket，再做新增的参考绑定授权检查，确保授权失败也能留下完整清理证据。我会补上这条反例后再交付；当前不需要你在板端操作。

11:31 UTC　补充反例与最终回归

补充反例已通过：新增授权检查失败时，仍保留原锁 ticket 和 closed_owner 清理证据。最终稿现在共 224 个不同测试方法，正在实际 ZIP 解包后的空目录中做最后复核。交审材料会明确区分原失败、前一作者稿和这份最终稿，避免混用哈希。

第一项修正处理模块引用间接保留 owner 的问题；第二项把锁 ticket 采集前移到新增授权检查之前，使授权失败也能留下清理证据。模型依据团队列定的四处路径实现并自检，团队再将候选交独立审查。最终 224 个不同测试方法通过、0 跳过；这组记录属于离线修补，后续实板成绩单列。来源：E40；KWS-pointwise-runtime-binding-20261024_02。

团队的判断与实现参与

团队把优化方向拆解到具体函数、参数和测试顺序。完整输出传递的调用示例明确了协调器的验收接口，四处 sources()清单限定了运行期接线覆盖；构建成员完成异机构建与来源核对，负责人组织现场验证并决定采用范围。模型据此编写候选、脚本和反例，保留实现错误及补测结果。